

L'objectif de cette séance est de se familiariser avec la spécification des programmes et les tests. La plupart des questions sont informelles et ne demandent pas de connaissances particulière vis-a-vis de la spécification, mais seulement une compréhension de la programmation.

Satisfaction des spécifications

On spécifie le programme `estPaire` comme suit : « Le programme prend en entrée un entier n et renvoie un booléen vallant `true` lorsque n est paire. » Parmi les programmes suivants, indiquer s'ils vérifient ou non la spécification donnée, et dans le cas où la spécification n'est pas vérifiée, donner un exemple d'une entrée sur laquelle le programme ne satisfait pas la spécification :

1.

```
bool estPaire (int n){  
    if (n%2 == 0){  
        return true;  
    }  
}
```

2.

```
bool estPaire (int n){  
    return true;  
}
```

3.

```
bool estPaire (int n){  
    if (n >=5){  
        return true;  
    }  
}
```

4.

```
bool estPaire (int n){  
    if (n % 4 == 0){  
        return true;  
    }  
    else if (n % 4 == 2){  
        return true;  
    }  
}
```

```
    else{  
        return false  
    }  
}
```

Spécification d'un algorithme de tri

1. Donner une spécification informelle (mais précise) d'un algorithme de tri sur des tableaux.
2. Donner une version formelle de cette spécification. Vous pouvez supposer que vous avez à votre dispositions toutes les fonctions mathématiques dont vous avez besoin pour extraire des informations sur les tableaux.
3. Proposer une série de tests pour cet algorithme, en indiquant à chaque fois l'objectif du test, les entrées proposées, et le résultat attendu.

Spécification de recherche dichotomique

Voici un programme qui implémente la recherche dichotomique dans un tableau.

```
int rechercheDichotomique (int t[], int i, int j, int a){  
    int m;  
    if (j = i+1){  
        return i;  
    }  
    m = (i+j)/2;  
    if (t[m] < a){  
        return rechercheDichotomique(t,i,m,a);  
    }  
    else{  
        return rechercheDichotomique(t,m,j,a);  
    }  
}
```

1. Donner une spécification informelle (mais précise) de cet algorithme. Pour cela, vous devrez :
 - (a) Penser aux conditions initiales sur le tableau **t**.
 - (b) Penser aux indices **i** et **j** par rapport à **t**, et les uns par rapport aux autres.
 - (c) Penser valeurs contenues dans **t** aux positions **i** et **j**.
 - (d) Penser à valeur **a**.
 - (e) Penser à la valeur de retour de la fonction.
2. Utiliser la spécification donnée pour vérifier le code de la recherche dichotomique.

Vérifier une spécification est difficile

On donne la spécification du programme `mystere` suivant : « Le programme `mystere` prend en entrée un entier `n` et renvoie toujours 1 ». Regarder et comprendre le programme suivant :

```
int mystere (int i){
    while(i != 1){
        if (i%2 == 0){
            i = i/2;
        } else {
            i = 3*i+1;
        }
    }
}
```

Décider si le programme vérifie sa spécification est la conjecture de syracuse. Il s'agit d'un problème mathématique auquel personne n'a de réponse.

S'assurer qu'un programme vérifie une spécification en général est donc un problème complexe : on ne peut pas espérer une solution simple qui couvre tous les cas